

PnP Ecommerce Service

Architecture Design Document

Java 17 · Spring Boot 3.2 · H2 · Spring Data JPA · Spring Security · Docker · GitHub Actions

1. System Layers

HTTP Clients	curl · Swagger UI · Postman
Web Layer	@RestController · @Valid · GlobalExceptionHandler · HTTP Basic Auth · Swagger UI
Service Layer	Business rules · @Transactional · stock checks · order state machine
Repository Layer	Spring Data JPA (CRUD) · NamedParameterJdbcTemplate (reporting SQL)
Database	H2 in-memory · schema.sql (named DDL constraints + indexes) · data.sql (10 seed products)

2. Domain Model

■ products		■ orders		■ order_items	
id	BIGINT PK	id	BIGINT PK	id	BIGINT PK
name	VARCHAR(255) NN	customer_name	VARCHAR(255) NN	order_id -> orders	BIGINT FK
description	CLOB	customer_email	VARCHAR(255) NN	product_id -> products	BIGINT FK
price cents (R39.99=3999)	INT NN >=0	status CONFIRMED CANCELLED	VARCHAR(20) NN	quantity	INT NN >0
stock_quantity	INT NN >=0	total_amount cents	BIGINT NN >=0	unit_price snapshot at order time	INT NN >=0
sku	VARCHAR(100) UQ	created_at	TIMESTAMP		
created_at	TIMESTAMP	updated_at	TIMESTAMP		
updated_at	TIMESTAMP				

Relationships

Order	1 ■■■■ ∞	OrderItem	One order → many items (cascade ALL, orphanRemoval=true)
Product	1 ■■■■ ∞	OrderItem	Product referenced across many order items (no cascade)

Indexes

idx_order_items_order_id · idx_order_items_product_id · idx_orders_status · idx_orders_customer_email

3. API Contract

■ Products — public reads, authenticated writes

POST	/api/v1/products Create product · validates name, price >=0, stock >=0, unique SKU	■ Basic Auth
GET	/api/v1/products?name=&minPrice;=&maxPrice;=&page;=&size;=&sort;= Paginated list · optional name substring + price range filter	

GET /api/v1/products/{id}
Single product · 404 if not found

■ Orders — public reads, authenticated writes

POST /api/v1/orders
Place order · stock check · atomic decrement · returns CONFIRMED order

■ Basic Auth

GET /api/v1/orders/{id}
Order with full item list and totals · 404 if not found

PATCH /api/v1/orders/{id}/cancel
Cancel CONFIRMED order · restores stock · 409 if already CANCELLED

■ Basic Auth

■ Reports — public reads

GET /api/v1/reports/top-products?limit=10&from;=&to;=
Top N products by units sold · CONFIRMED only · optional ISO date range

GET /api/v1/reports/order-summary
Order counts, total revenue, avg order value grouped by status

Consistent error response shape

```
{ "status": 404, "error": "NOT_FOUND", "message": "Product with ID 99 was not found", "path":  
"/api/v1/products/99", "timestamp": "...", "fieldErrors": null }
```

4. Key Design Decisions

Prices in cents	R39.99 stored as 3999 (INT). Eliminates IEEE 754 floating-point rounding errors. Formatted as R%.2f at response time only.
Pessimistic write lock	findByIdWithLock() uses @Lock(PESSIMISTIC_WRITE) — issues SELECT ... FOR UPDATE. Prevents concurrent requests from both seeing sufficient stock and both proceeding, causing negative inventory.
Price snapshot on OrderItem	unit_price captured at order placement time. Future catalogue price changes cannot alter historical order totals.
Explicit schema.sql	ddl-auto=none — Hibernate never generates DDL. All tables, named constraints (pk_/uq_/fk_/chk_), and indexes are declared in schema.sql, making schema design explicitly visible to reviewers.
JPA + JdbcTemplate mix	Spring Data JPA for CRUD entities. NamedParameterJdbcTemplate for reporting — keeps JOIN/GROUP BY SQL readable and explicit rather than buried in JPQL.
Security from day 1	SecurityConfig wired in Iteration 1. GET=public, POST+PATCH=authenticated. BCrypt + InMemoryUserDetailsManager. Credentials injected via @Value for environment variable override.
CI from first commit	ci.yml created in Iteration 1 — runs mvn -B verify on every push and PR. Early iterations get a compile check; Iteration 5 adds tests and CI picks them up with no config change.
H2 in-memory database	Self-contained — no database setup for reviewers. Schema and seed data load on startup. H2 console at /h2-console for live inspection.

5. Incremental Build Plan

1 Scaffold · Domain · Schema · Security · CI

→ →	Spring Boot 3 project — Maven, Java 17, H2, JPA, Validation, Lombok, Springdoc
→ →	Product, Order, OrderItem entities + OrderStatus enum
→ →	schema.sql — named FK/CHECK constraints, 4 indexes · data.sql — 10 seed products
→ →	application.properties — ddl-auto=none, H2 console, Swagger path, credentials
→ security	SecurityConfig — HTTP Basic, stateless, GET=public / POST+PATCH=authenticated, BCrypt, InMemoryUserDetailsManager
→ security	OpenApiConfig — Swagger UI with basicAuth scheme and Authorize button
→ ci	.github/workflows/ci.yml — mvn -B verify on push/PR, Maven cache, Surefire upload on failure
✓ App starts · H2 tables populated · unauthenticated POST returns 401 · CI green from first commit	

2 Product Catalogue APIs

→ →	ProductRepository — JPQL search (name/price filters), pessimistic lock method, existsBySku
→ →	ProductService + ProductServiceImpl — create, getProducts paginated, getById
→ →	ProductController (POST, GET list, GET /{id}) + request/response DTOs
→ →	GlobalExceptionHandler — ResourceNotFoundException (404), DuplicateSkuException (409), validation (400)
✓ Paginated product list · filter by name + price range · create requires auth · 404 on missing ID	

3 Order Placement + Cancellation

→ →	OrderRepository
→ →	OrderServiceImpl — placeOrder: lock → stock check → decrement → snapshot unit_price
→ →	cancelOrder — CONFIRMED check → restore stock per item → set CANCELLED
→ →	OrderController (POST, GET /{id}, PATCH /{id}/cancel) + DTOs
→ →	InsufficientStockException (409), InvalidOrderStateException (409) added to GlobalExceptionHandler
✓ Place order deducts stock · cancel restores it · 409 on bad state or insufficient stock	

4 Reporting Endpoint

→ →	ReportRepository — NamedParameterJdbcTemplate with two explicit SQL queries
→ →	top-products: 3-table JOIN → GROUP BY → SUM/COUNT(DISTINCT) → date-range filter → LIMIT N · CONFIRMED only
→ →	order-summary: GROUP BY status → COALESCE SUM → AVG cast to BIGINT
→ →	ReportService + ReportController + response DTOs
✓ Top-N products ranked by units sold · order summary with counts + revenue by status	

5 Tests + Docker

→ →	@DataJpaTest — ProductRepositoryTest (filter, pagination, stock helpers)
→ →	Mockito — ProductServiceTest, OrderServiceTest (all happy + all error branches)
→ →	@WebMvcTest — ProductControllerTest, OrderControllerTest (HTTP status + validation + @WithMockUser)
→ →	@SpringBootTest — ReportControllerIntegrationTest (@DirtiesContext, real SQL against H2)
→ docker	Multi-stage Dockerfile — Maven build stage + JRE runtime (eclipse-temurin:17-jre-alpine), non-root user
→ docker	docker-compose.yml — credential env vars, health check on GET /api/v1/products
✓ mvn test passes · docker compose up --build starts healthy service · CI runs full suite on push	

CI lifecycle: ci.yml created in Iteration 1 gives a compile check from the first commit. When tests are added in Iteration 5, CI automatically runs the full suite — no config change needed.